DISEÑO DIGITAL 1.

TÍTULO DE LA ACTIVIDAD: **Modelado y simulación de autómatas**	**CÓDIGO:** **AIP8**

FECHA:			
NOMBRE:		APELLIDOS:	

MODALIDAD:	Individual. Tutorial.	TIPO:	Presencial	DURACIÓN:	50 minutos

CALENDARIO:	Sesión presencial S10	REQUISITOS:	Lectura sobre modelado de autómatas en VHDL.

CRITERIO DE ÉXITO:	

COMENTARIOS E INCIDENCIAS:

TIEMPO DEDICADO:	minutos	AUTOEVALUACIÓN: [entre 0 y 10 puntos]	No procede

Introducción

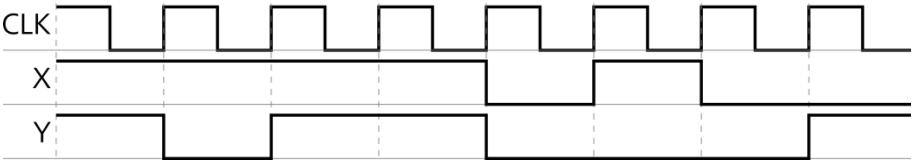
En esta actividad debe diseñar el diagrama de estados, codificar un modelo en VHDL y verificar el funcionamiento mediante simulación de un autómata que multiplica un número por tres. A lo largo de la actividad aprenderá cómo analizar los pasos de un algoritmo para averiguar qué información de estado es necesaria y verá que a veces aparecen estados redundantes al diseñar un diagrama de estados.

Multiplicador por tres

Especificación

Se desea modelar un autómata que multiplique por tres un número codificado en binario natural que recibe por su entrada *X*, bit a bit, en sincronía con la señal de reloj, comenzando por el bit menos significativo. La longitud del número puede ser cualquiera, aunque se garantiza que los dos bits de mayor peso serán siempre cero. El resultado se saca también en serie por su salida *Y*, comenzando igualmente por el bit menos significativo.

Por ejemplo si, como ocurre en el cronograma de la figura, por la entrada se recibe la secuencia "11110100", que corresponde al número 00101111_B (47), la secuencia de salida debe ser "10110001" (ver figura), ya que $47 \cdot 3 = 141 = 10001101_B$.



Obtención del diagrama de estados

El cronograma de ejemplo que se proporciona en la especificación indica que el bit de menor peso de la salida se obtiene en el mismo ciclo de reloj en que se presenta en la entrada el bit de menor peso del operando, por lo que debe utilizarse la arquitectura de Mealy.

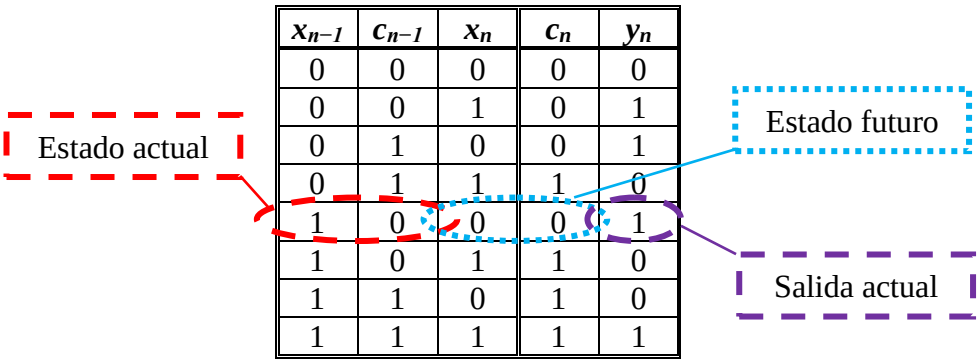
En cualquier caso, antes de empezar a dibujar el diagrama de estados es preciso estudiar el algoritmo que se empleará para realizar la operación requerida, y a partir de él deducir qué información de estado habrá que guardar. En muchos casos el propio algoritmo puede no ser único, por lo que habrá que elegir entre las posibilidades que haya.

En este ejemplo puede realizarse la multiplicación mediante una suma, teniendo en cuenta que $3 \cdot x = 2 \cdot x + x$, y que $2 \cdot x$ se obtiene, cuando x está codificado en binario natural, desplazando x un bit hacia la izquierda y rellenando con un cero por la derecha. Por tanto el producto se puede calcular con la siguiente suma:

$$\begin{array}{cccccc} & \ldots & x_3 & x_2 & x_1 & x_0 \\ + & \ldots & x_2 & x_1 & x_0 & 0 \\ \hline & \ldots & y_3 & y_2 & y_1 & y_0 \end{array}$$

donde x_n representa el bit n de x. Evidentemente $y_0 = x_0$ e $y_1 = x_1 + x_0$. Fíjese en que esta suma ya puede producir un acarreo, al que llamaremos c_1, si x_1 y x_0 valen ambos 1. A partir de ahí se puede generalizar c_n, $y_n = x_n + x_{n-1} + c_{n-1}$, y queda claro que la operación que hay que realizar para calcular cada bit de salida es una suma completa (es decir, con acarreos de entrada y salida) de un bit. Fíjese en que por definición $c_{-1} = 0$ (no hay acarreo de entrada en la suma de los bits menos significativos) y que $c_0 = 0$ (ya que $y_0 = x_0 + 0 = x_0$).

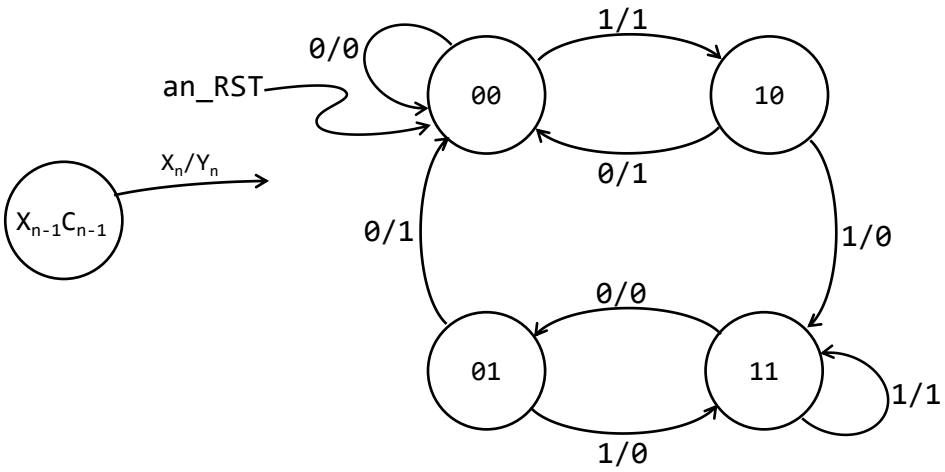
Viendo esto, y dado que x_n está disponible como entrada, parece que la información de estado necesaria en cada momento son los bits x_{n-1} y c_{n-1}, lo que daría lugar a un autómata con cuatro estados. En cada estado se calcula la suma de tres bits: x_n es el bit presente en la entrada, y x_{n-1} y c_{n-1} son los que están almacenados como información de estado. La salida en cada estado es el resultado de la suma, y_n, y los bits x_n y c_n se "almacenan" en el estado siguiente. Toda esta información se puede ver en la tabla de verdad de un sumador completo de un bit:



x_{n-1}	c_{n-1}	x_n	c_n	y_n
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

En la tabla está marcado un ejemplo en el que la entrada actual (x_n) es "0", la entrada en el ciclo de reloj anterior (x_{n-1}) fue "1" y la suma de ese ciclo no produjo acarreo (c_{n-1} = "0"). La salida actual (y_n) es "1" y el acarreo actual (c_n) es "0", ya que 1 + 0 + 0 = 1. La información que habrá que guardar en el estado siguiente es la entrada y el acarreo actuales (x_n y c_n), y eso es por tanto lo que constituye el estado futuro.

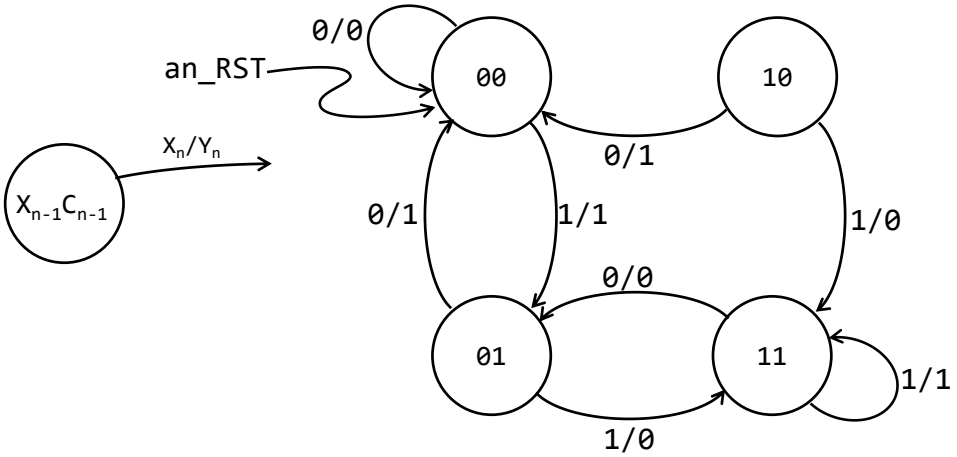
El diagrama de estados al que da lugar esta tabla es el siguiente:



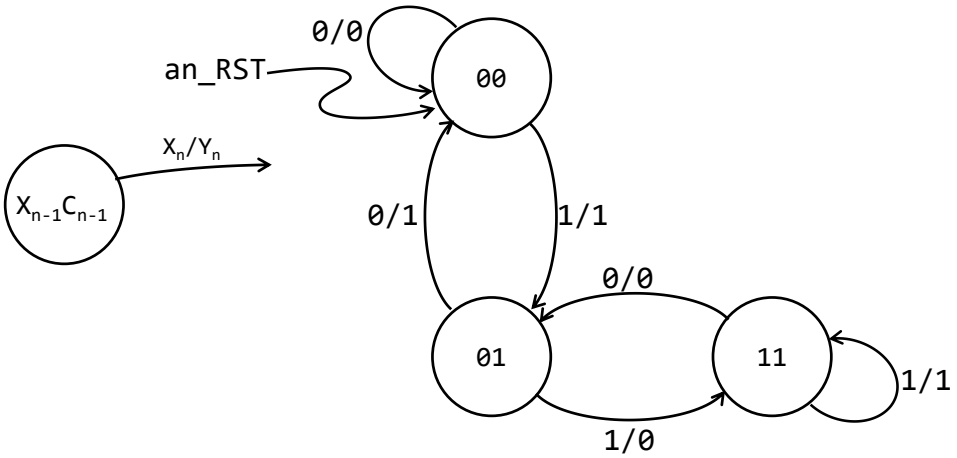
Fíjese, sin embargo, en un detalle interesante: los estados "01" y "10" producen la misma salida ante la misma entrada (la salida es "1" si la entrada es "0", y "0" si la entrada es "1"), y ambos conducen

al mismo estado para entradas iguales (al estado “00” si la entrada es “0”, y al estado “11” si la entrada es “1”).

Esto implica que si, por ejemplo, cambiamos en el diagrama de estados la transición que lleva del estado “00” al estado “10” cuando la entrada es “1” de modo que lleve al estado “01”, tal como se muestra en la siguiente figura, el autómata resultante es equivalente al autómata original, ya que ambos producen la misma secuencia de salida ante una determinada secuencia de entrada.



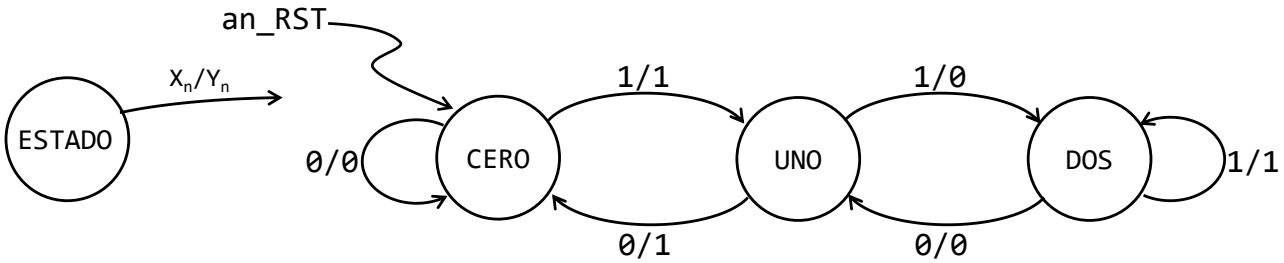
Fíjese en que en el nuevo diagrama de estados hay un estado (el “10”) al que no llega ninguna transición. Es, por tanto, un estado sobrante, ya que es imposible que el autómata llegue a ese estado. Puede (debe, de hecho) eliminarse, quedando el diagrama de estados como sigue:



¿Cómo es posible que el nuevo autómata sea equivalente al original teniendo un estado menos? La respuesta es que en el primer diagrama de estados existen *estados redundantes*, es decir, estados duplicados que pueden, como se ha demostrado, eliminarse sin afectar al comportamiento del autómata. La existencia de estados redundantes implica que la información de estado es en sí misma redundante, es decir, que se almacena más información de estado de la que es estrictamente necesaria para que el autómata funcione correctamente.

¿De dónde viene esa redundancia en la información de estado? En este caso viene de que se guardan dos dígitos que pueden tomar en conjunto cuatro valores distintos, cuando la única información que se necesita en realidad es *la suma* de esos dos dígitos, que sólo puede tomar tres valores (0, 1 ó 2) en lugar de cuatro.

La siguiente figura muestra una versión reorganizada del diagrama de estados simplificado en la que los nombres de los estados corresponden al valor de la suma $x_{n-1} + c_{n-1}$.



Aunque en este ejemplo la detección de estados redundantes era sencilla (existían dos estados que generaban la misma salida y conducían a los mismos estados ante las mismas entradas), en general la existencia de estados redundantes no tiene por qué ser evidente. Existen técnicas formales para detectar y eliminar estados redundantes en un autómata, pero su estudio está fuera de los objetivos de esta asignatura. Baste por el momento conocer que es posible que un autómata tenga estados (o grupos de estados) redundantes, que esto es en general indeseable porque aumenta el tamaño de la memoria de estado y la complejidad del autómata de forma innecesaria, y que existen técnicas formales para detectar y corregir este problema.

Modelado en VHDL y simulación

Cree un proyecto ModelSim con el nombre *Proy_AIP8*. Codifique el modelo VHDL del autómata multiplicador por tres que se ha descrito en el apartado anterior, guardándolo con el nombre *mult3serie.vhd*. Codifique un banco de test, guárdelo con el nombre *tb_mult3serie.vhd*, y simule el sistema para asegurarse de que el funcionamiento es correcto.